



Introduction to Database Systems

2024-Fall



5. The Security and Integrity Constraints



Introduction

- The destruction of database is generally caused by the following factors:
 - System failure
 - Inconsistency caused by concurrent access
 - Man-caused destruction (intentionally or accidentally)
 - The data inputted is incorrect, the updating transaction didn't obey the rule of consistency preservation
- In above factors, 1 and 2 should be resolved by recovery mechanism of DBMS (Chapter 4); 3 belongs to database security; 4 belongs to integrity constraints



Introduction to DB Security

- **Secrecy**: Users should not be able to see things they are not supposed to.
 - E.g., A student can't see other students' grades.
- **Integrity**: Users should not be able to modify things they are not supposed to.
 - E.g., Only instructors can assign grades.
- **Availability**: Users should be able to see and modify things they are allowed to.



Security of Database

- Protect databases not be accessed illegally.
 - *View and query rewriting*
 - *Access control*
 - Access control is the management of users' permissions to access various resources in a database.
 - Resources: base table, view, catalog, applications
 - Permissions: create, delete, query, add, update...
 - Database Users:
 - General users;
 - Database users with authority over certain resources;
 - DBA privileges.



Access Controls

- ***Identification and authentication of users***
 - Password
 - Special articles, such as key, IC card, etc.
 - Personal features, such as fingerprint, signature, etc.
- ***Role***
 - When a role is granted to a user, that user gains the permissions associated with the role.
 - A user can have multiple roles and other permissions.
- Two main mechanisms at the DBMS level:
 - Discretionary access control
 - Mandatory access control



Discretionary Access Control

- Based on the concept of access rights or ***privileges*** for objects (tables and views), and mechanisms for giving users privileges (and revoking privileges).
- Creator of a table or a view automatically gets all privileges on it.
- DMBS keeps track of who subsequently gains and loses privileges, and ensures that only requests from users who have the necessary privileges (at the time the request is issued) are allowed.



GRANT Command

GRANT privileges **ON** object **TO** users [**WITH GRANT OPTION**]

- The following *privileges* can be specified:
 - SELECT: Can read all columns (including those added later via ALTER TABLE command).
 - INSERT(col-name): Can insert tuples with non-null or non-default values in this column.
 - INSERT means same right with respect to all columns.
 - DELETE: Can delete tuples.
 - REFERENCES (col-name): Can define foreign keys (in other tables) that refer to this column.
- If a user has a privilege with the GRANT OPTION, can pass privilege on to other users (with or without passing on the GRANT OPTION).
- ***Only owner can execute CREATE, ALTER, and DROP.***



GRANT and REVOKE of Privileges

- GRANT INSERT, SELECT ON Sailors TO Horatio
 - Horatio can query Sailors or insert tuples into it.
- GRANT DELETE ON Sailors TO Yuppy WITH GRANT OPTION
 - Yuppy can delete tuples, and also authorize others to do so.
- GRANT UPDATE (*rating*) ON Sailors TO Dustin
 - Dustin can update (only) the *rating* field of Sailors tuples.
- GRANT SELECT ON ActiveSailors TO Guppy, Yuppy
 - This does NOT allow the Guppy, Yuppy to query Sailors directly!
- **REVOKE**: When a privilege is revoked from X, it is also revoked from all users who got it *solely* from X.
 - **REVOKE SELECT ON** ActiveSailors **FROM** Guppy;
 - **REVOKE ALL ON** ActiveSailors **FROM** Guppy;



GRANT/REVOKE on Views

- If the creator of a view loses the SELECT privilege on an underlying table, the view is dropped!
- If the creator of a view loses a privilege held with the grant option on an underlying table, (s)he loses the privilege on the view as well; so do users who were granted that privilege on the view!



Views and Security

- Views can be used to present necessary information (or a summary), while hiding details in underlying relation(s).
 - Given Active Sailors, but not Sailors or Reserves, we can find sailors who have a reservation, but not the *bid's* of boats that have been reserved.
- Creator of view has a privilege on the view if (s)he has the privilege on all underlying tables.
- Together with GRANT/REVOKE commands, views are a very powerful access control tool.



Role-Based Authorization

- In SQL-92, privileges are actually assigned to authorization ids, which can denote a single user or a group of users.
- In SQL:1999 (and in many current systems), privileges are assigned to roles.
 - Roles can then be granted to users and to other roles.
 - Reflects how real organizations work.
 - Illustrates how standards often catch up with “de facto” standards embodied in popular systems.



Security to the Level of a Field!

- Can create a view that only returns one field of one tuple. (How?)
- Then grant access to that view accordingly.
- Allows for *arbitrary* granularity of control, *but*:
 - Clumsy to specify, though this can be hidden under a good UI
 - Performance is unacceptable if we need to define field-granularity access frequently. (Too many view creations and look-ups.)



Internet-Oriented Security

- Key Issues: User authentication and trust.
 - When DB must be accessed from a secure location, password-based schemes are usually adequate.
- For access over an external network, trust is hard to achieve.
 - If someone with Sam's credit card wants to buy from you, how can you be sure it is not someone who stole his card?
 - How can Sam be sure that the screen for entering his credit card information is indeed yours, and not some rogue site spoofing you (to steal such information)? How can he be sure that sensitive information is not "sniffed" while it is being sent over the network to you?
- **Encryption** is a technique used to address these issues.



Encryption

- “Masks” data for secure transmission or storage
 - $\text{Encrypt}(\text{data}, \text{encryption key}) = \text{encrypted data}$
 - $\text{Decrypt}(\text{encrypted data}, \text{decryption key}) = \text{original data}$
 - Without decryption key, the encrypted data is meaningless gibberish
- Symmetric Encryption:
 - Encryption key = decryption key; all authorized users know decryption key (a weakness).
 - DES, used since 1977, has 56-bit key; AES has 128-bit (optionally, 192-bit or 256-bit) key
- **Public-Key Encryption:** Each user has two keys:
 - User’s public encryption key: Known to all
 - Decryption key: Known only to this user
 - Used in RSA scheme (Turing Award!)



Mandatory Access Control

- Based on system-wide policies that cannot be changed by individual users.
 - Each DB object is assigned a security class.
 - Each subject (user or user program) is assigned a clearance for a security class.
 - Rules based on security classes and clearances govern who can read/write which objects.
- Most commercial systems do not support mandatory access control. Versions of some DBMSs do support it; used for specialized (e.g., military) applications.



Why Mandatory Control?

- Discretionary control has some flaws, e.g., the *Trojan horse* problem:
 - Dick creates a table and gives INSERT privileges to Justin (who doesn't know about this).
 - Dick modifies the code of an application program used by Justin to additionally write some secret data to the table.
 - Now, Justin can see the secret info.
- The modification of the code is beyond the DBMSs control, but it can try and prevent the use of the database as a channel for secret information.



Security of Database cont.

- ***Protect databases not be accessed illegally.***
 - Authorization
 - GRANT CONNECT TO JOHN IDENTIFIED BY xyzabc;
 - GRANT SELECT ON TABLE S TO U1 WITH GRANT OPTION;
 - Role
 - Data encryption
 - ***Audit trail***
 - AUDIT SELECT, INSERT, DELETE, UPDATE ON emp WHENEVER SUCCESSFUL;





Security of Statistical Database

- In many situation, the statistical data is public while the detailed individual data is secret.
- Public *statistical database*
- But some detailed individual data can be derived from public statistical data
 - How prevent this leak? --- not a easy thing



Security of Statistical Database

STATS

NAME	SEX	DEPENDENTS	OCCUPATION	SALARY
Wang	M	2	Programmer	120
Chang	F	2	Manager	240
Chen	F	0	Programmer	140
Li	F	2	Engineer	160
Liu	M	2	Clerk	110
Zhu	F	1	Teacher	80
Zhao	M	0	Professor	180
Sun	M	1	Teacher	110
Xu	F	2	Programmer	130
Ma	F	1	Programmer	150





Individual Tracker

- Suppose we know Wang is a male programmer, and salary in STATS is secret but other information is public, we can get wang's salary from public data.

- Q1: SELECT COUNT(*)
FROM STATS
WHERE SEX='M' AND OCCUPATION='programmer'

result = 1

- Q2: SELECT SUM(SALARY)
FROM STATS
WHERE SEX='M' AND OCCUPATION='programmer';

result = 120



Individual Tracker ($c > b$, $b = 2$)

- Q3: SELECT COUNT(*)
FROM STATS;

result = 10

Q4: SELECT COUNT(*)
FROM STATS

WHERE NOT(SEX='M' AND OCCUPATION='programmer');

result = 9

Now we know only one male programmer, that must be Wang.



Individual Tracker ($c > b$, $b = 2$)

- Q5: SELECT SUM(SALARY)
FROM STATS;

result = 1420

Q4: SELECT SUM(SALARY)
FROM STATS

WHERE NOT(SEX='M' AND OCCUPATION='programmer');

result = 1300

Wang's salary = Q5 - Q6 = 120



Individual Tracker ($b < c < n - b$, $b = 2$, n is 10)

- Q7: SELECT COUNT(*)
FROM STATS
WHERE SEX = 'M';

result = 4

Q8: SELECT COUNT(*)
FROM STATS
WHERE SEX='M' AND NOT(OCCUPATION='programmer');

result = 3

Now we know only one male programmer, that must be Wang.



Individual Tracker ($b < c < n - b$, $b=2$, n is 10)

- Q9: SELECT SUM(SALARY)
FROM STATS
WHERE SEX = 'M';

result = 520

Q10: SELECT SUM(SALARY)
FROM STATS
WHERE SEX='M' AND NOT(OCCUPATION='programmer');

result = 400

Wang's salary = $Q9 - Q10 = 120$



General Tracker

- ***Individual tracker***

- Suppose predicate $p=p_1$ and p_2 , $SET(p)$ is set of tuples which fulfill p , then
 - **$SET(p) = SET(p_1 \text{ and } p_2) = SET(p_1) - SET(p_1 \text{ and not } p_2)$**

- ***General tracker***

- It is a predicate T which fulfill:
 $2b \leq |SET(T)| \leq (n-2b), b < n/4$
- Suppose a tuple R can be limited uniquely by predicate p , that is $SET(p) = \{R\}$, then
 $SET(p) = SET(p \text{ or } T) \bar{\cup} SET(p \text{ or not } T) - SET(T) - SET(\text{not } T)$
- $\bar{\cup}$ means ***union all*** (without eliminating repeated tuples).



Integrity Constraints



Integrity Constraints

- An IC describes conditions that every *legal instance* of a relation must satisfy.
 - Inserts/deletes/updates that violate IC's are disallowed.
 - Can be used to ensure application semantics (e.g., *sid* is a key), or prevent inconsistencies (e.g., *sname* has to be a string, *age* must be < 200)



Types of Integrity Constraints

- **Static constraints:** constraints to database state
 - Inherent constraints (固有约束) (data model), such as **1NF**
 - Implicit constraints (隐含约束) : implied in data schema, indicated by DDL generally. Such as domain constraints, primary key constraints, foreign key constraints.
 - **Domain constraints:** Field values must be of right type. Always enforced.
 - Explicit constraints (显式约束) or general constraints
- **Dynamic constraints:** constraints while database transferring from one state to another. Can be combined with trigger.



An Example of Foreign Keys

- E.g. *sid* is a foreign key referring to **Students**:
 - Students(*sid*: string, *name*: string, *login*: string, *age*: integer, *gpa*: decimals)
 - Enrolled(*sid*: string, *cid*: string, *grade*: string)

Enrolled

<i>sid</i>	cid	grade
53666	Carnatic101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

Students

<i>sid</i>	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8



Database Modification

- If α is foreign key in r_2 which references to K_1 in r_1 , the following tests must be made in order to preserve the following referential integrity constraint:

$$\Pi_{\alpha}(r_2) \subseteq \Pi_{K_1}(r_1)$$

- **Insert.** If a tuple t_2 is inserted into r_2 , the system must ensure that there is a tuple t_1 in r_1 such that $t_1[K_1] = t_2[\alpha]$. That is

$$t_2[\alpha] \in \Pi_{K_1}(r_1)$$

- **Delete.** If a tuple, t_1 is deleted from r_1 , the system must compute the set of tuples in r_2 that reference t_1 :

$$\sigma_{\alpha = t_1[K_1]}(r_2)$$

If this set is not empty, either the delete command is **rejected as an error**, or the tuples that reference t_1 must themselves be deleted (**cascading deletions** (级联删除) are possible).



Database Modification (Cont.)

- **Update.** There are two cases:
 - If a tuple t_2 is updated in relation r_2 and the update modifies values for foreign key α , then a test similar to the insert case is made. Let t_2' denote the new value of tuple t_2 . The system must ensure that

$$t_2'[\alpha] \in \Pi_{K_1}(r_1)$$

- If a tuple t_1 is updated in r_1 , and the update modifies values for the primary key (K_1), then a test similar to the delete case is made. The system must compute

$$\sigma_{\alpha = t_1[K_1]}(r_2)$$

using the old value of t_1 (the value before the update is applied). If this set is not empty, the update may be **rejected as an error**, or the **update may be cascaded** to the tuples in the set, or the tuples in the set may be **deleted**.



Types of Integrity Constraints

- Static constraints: constraints to database state
 - Inherent constraints (data model), such as 1NF
 - Implicit constraints : implied in data schema, indicated by DDL generally. Such as domain constraints, primary key constraints, foreign key constraints.
 - *Domain constraints*: Field values must be of right type. Always enforced.
 - ***Explicit constraints or general constraints***
- Dynamic constraints: constraints while database transferring from one state to another. Can be combined with trigger.



Definition of Integrity Constraints

- **Indicated with *procedure***
 - Let application programs responsible for the checking of integrity constraints.
- **Indicated with *ASSERTION***
 - Defined with *assertion specification language*, and checked by DBMS automatically
 - ASSERT balanceCons ON account: balance $\geq$ 0;
 - Indicated with **CHECK** clause in base table definition, and checked by DBMS automatically
- **Trigger**
 - A trigger is a software mechanism, in the form of:
whenever <condition> then <action>



General Constraints

- Useful when more general ICs than keys are involved.
 - Can use queries to express constraint.
 - Constraints can be named.

```
CREATE TABLE Reserves
  ( sname CHAR(10),
    bid INTEGER,
    day DATE,
    PRIMARY KEY (bid, day),
    CONSTRAINT noInterlakeRes
    CHECK ('Interlake' <>
          (SELECT B.bname
           FROM Boats B
           WHERE B.bid=bid)))
```

```
CREATE TABLE Sailors
  ( sid INTEGER,
    sname CHAR(10),
    rating INTEGER,
    age REAL,
    PRIMARY KEY (sid),
    CHECK ( rating >= 1
           AND rating <= 10))
```



General Constraints

- Useful when more general ICs than keys are involved.
 - Can use queries to express constraint.
 - Constraints can be named.

```
create table Teachers(  
    ...  
    Age int check(age>10 and age<30),  
    ...  
)
```



General Constraints

- Use alter table clause to **add** CHECK constraints:
alter table [Users]
add constraint [CK_Users_Balance] check (Balance > 0)
- Use alter table clause to **drop** CHECK constraints:
alter table Teachers
drop constraint ck_age



Constraints Over Multiple Relations

```
CREATE TABLE Sailors
( sid INTEGER,
  sname CHAR(10),
  rating INTEGER,
  age REAL,
  PRIMARY KEY (sid),
```

CHECK

```
( (SELECT COUNT (S.sid) FROM Sailors S)
 +(SELECT COUNT (B.bid) FROM Boats B) < 100 )
```

*Number of boats
plus number of
sailors is < 100*

- **Awkward and wrong!**
- If Sailors is empty, the number of Boats tuples can be anything!



Assertion

- **ASSERTION** is the right solution; not associated with either table.

```
CREATE ASSERTION smallClub  
CHECK  
( (SELECT COUNT (S.sid) FROM Sailors S)  
+(SELECT COUNT (B.bid) FROM Boats B) < 100 )
```



Assertion

1. 定义条件:

断言通过一个 `CREATE ASSERTION` 语句在数据库中定义。这一语句包含一个逻辑条件（通常使用 SQL 的 `CHECK` 子句来表达），描述了数据库必须始终满足的约束。

2. 自动检查:

数据库管理系统 (DBMS) 会在数据插入、更新或删除操作时自动检查这些断言。当任何操作试图改变数据库中的数据时，DBMS 会验证断言所定义的条件。

3. 维护完整性:

如果某个操作导致断言的条件不再成立，DBMS 将拒绝该操作，通常会返回一个错误信息。这种机制有效地防止了不符合业务规则的数据进入数据库。

4. 全局约束:

断言可以用于定义跨多个表的复杂约束，而不仅仅局限于单个表。它可以确保多个表之间的数据一致性。



Different Constraint Types

Type	Where Declared	When activated	Guaranteed to hold?
Attribute CHECK	with attribute	on insertion or update	not if contains subquery
Tuple CHECK	relation schema	insertion or update to relation	not if contains subquery
Assertion	database schema	on change to any relation mentioned	Yes

根据SQL标准和大多数数据库管理系统（DBMS）的实现，CHECK 约束内通常不允许使用子查询。



Triggers

- Trigger: procedure that starts automatically if specified changes occur to the DBMS
- Three parts:
 - **E**vent (activates the trigger)
 - **C**ondition (tests whether the triggers should run)
 - **A**ction (what happens if the trigger runs)



Triggers: Example (SQL:1999)

```
CREATE TRIGGER youngSailorUpdate
  AFTER INSERT ON SAILORS
  REFERENCING NEW TABLE NewSailors
  FOR EACH STATEMENT
  INSERT
    INTO YoungSailors(sid, name, age, rating)
    SELECT sid, name, age, rating
    FROM NewSailors N
    WHERE N.age <= 18
```



Triggers

- Synchronization of the Trigger with the activating statement (DB modification)
 - Before
 - After
 - Instead of
 - An INSTEAD OF trigger is a type of database trigger that allows you to define **custom behavior** to be executed instead of a standard INSERT, UPDATE, or DELETE operation. This is particularly useful for **views**, which are virtual tables derived from a query on one or more underlying tables.

--Create INSTEAD OF trigger

```
CREATE TRIGGER UpdateEmployeeView  
INSTEAD OF UPDATE ON EmployeeView    referencing new Table NEW  
FOR EACH ROW  
BEGIN
```

-- Update the Employees table

```
UPDATE Employees  
SET name = NEW.name,  
      department_id = (SELECT department_id FROM Departments WHERE department_name =  
NEW.department_name)  
      WHERE employee_id = OLD.employee_id;  
END;
```

- Deferred (at end of transaction).



Triggers

- Synchronization of the Trigger with the activating statement (DB modification)
 - Before
 - After
 - Instead of
 - Deferred (at end of transaction).
- **Update events** may specify a particular column or set of columns.
- A **condition** is specified with a WHEN clause.
- Number of **Activations** of the Trigger
 - Once per modified tuple
(FOR EACH ROW)
 - Once per activating statement
(default).



Triggers

- Options for the **REFERENCING** clause:
 - **NEW TABLE**: the set (!) of tuples newly inserted (INSERT).
 - **OLD TABLE**: the set (!) of deleted or old versions of tuples (DELETE / UPDATE).
 - **OLD ROW**: the old version of the tuple (FOR EACH ROW UPDATE).
 - **NEW ROW**: the new version of the tuple (FOR EACH ROW UPDATE).
- The action of a trigger can consist of multiple SQL statements, surrounded by **BEGIN . . . END**.



Example: Row Level Trigger

```
CREATE TRIGGER NoLowerPrices
AFTER UPDATE OF price ON Product
REFERENCING
    OLD AS OldTuple
    NEW AS NewTuple
WHEN (OldTuple.price > NewTuple.price)
UPDATE Product
SET price = OldTuple.price
WHERE name = NewTuple.name
FOR EACH ROW
```



Statement Level Triggers

- Instead of executing a separate action for each affected row, a single action can be executed for all rows affected by a transaction
 - Use **for each statement** instead of **for each row**
 - Use **referencing old table** or **referencing new table** to refer to temporary tables (called *transition tables*) containing the affected rows
 - Can be more efficient when dealing with SQL statements that update a large number of rows



Triggers: Example

```
CREATE TRIGGER youngSailorUpdate
AFTER INSERT ON SAILORS                /* Event */
REFERENCING NEW TABLE NewSailors
FOR EACH STATEMENT
INSERT                                /* Action */
    INTO YoungSailors(sid, name, age, rating)
    SELECT sid, name, age, rating
    FROM NewSailors N
    WHERE N.age <= 18
```

- This trigger inserts young sailors into a separate table.
- It has no (i.e., an empty, always true) condition.



Triggers: Example

```
CREATE TRIGGER notTooManyReservations
  AFTER INSERT ON Reserves                                /* Event */
  REFERENCING NEW ROW NewReservation
  FOR EACH ROW
  WHEN (10 <= (SELECT COUNT(*) FROM Reserves
               WHERE sid =NewReservation.sid))           /* Condition */
  DELETE FROM Reserves R
  WHERE R.sid= NewReservation.sid                          /* Action */
        AND day=
        (SELECT MIN(day) FROM Reserves R2 WHERE R2.sid=R.sid);
```

- This trigger makes sure that a sailor has less than 10 reservations, deleting the oldest reservation of a given sailor, if necessary.
- It has a non-empty condition (**WHEN**).



Triggers vs. General Constraints

- Triggers can be harder to understand.
 - Several triggers can be activated by one SQL statement (*arbitrary order!*).
 - A trigger may activate other triggers (*chain activation*).
- Triggers are procedural.
 - Assertions react on any database modification, trigger only specified event.
 - *Trigger execution cannot be optimized by DBMS.*
- Triggers have more applications than constraints.
 - monitor integrity constraints,
 - construct a log,
 - gather database statistics, etc.





Assuming the following three relations exist:

Sailors(sid, sname, rating, age, master)

/* where sid is the sailor's ID, sname is the name, rating is the level, age is the birth date, and master is the ID of the sailor's master, who is also a sailor */

Boats(bid, bname, color)

/ where bid is the boat's ID, bname is the name, and color is the color of the boat
/

Reserves(sid, bid, day)

/ where sid is the ID of the sailor who reserved a boat, bid is the ID of the reserved boat, and day is the date of reservation */



CASE

Implementation of Referencing Integrity Rules

Taking Sailors, Boats, and Reserves as examples, write the rules for implementing referential integrity constraints.

What actions can affect the referential integrity between the three tables?

- Reserves Insert
- Sailors Delete
- Boats Delete
- Reserves Update
- Sailors Update
- Boats Update

Does the Update operation on all attributes affect referential integrity?



Rule 1

Create a trigger to monitor the **Insert operation** of the Reserves table. *If* the foreign key attribute of the inserted tuple does not exist in the Sailors and Boats tables, **roll back** the operation of inserting the record.

Create trigger referential_integrity_check

Before Insert on Reserves

Referencing NEW as N

For Each Row

When (not (exists(Select * From Sailors Where sid = N.sid)
and
(exists(Select * From Boats Where bid = N.bid))
)

Rollback;

Action

Condition



Rule 2

Create a trigger to monitor the **Delete operation** on the Boats table. *If* the primary key of the deleted tuple is a foreign key in the Reserves table, **roll back** the operation to delete the record.

Create trigger boats_delete

Before Delete on Boats

Referencing OLD as O

For Each Row

**When (exists(Select * From Reserves
Where bid = O.bid))**

Rollback;

Event

Action

Condition



Rule 3

Create a trigger to monitor the **Delete operation** on the Sailors table. *If* the primary key of the deleted tuple is a foreign key in the Reserves table, the relevant records in the Reserves table will be *deleted*.

Create trigger sailors_delete

After Delete on Sailors

Event

Referencing OLD as O

For Each Row

When (exists(Select * From Reserves
Where sid = O.sid))

Delete From Reserves
Where sid = O.sid;

Condition

Action



Rule 4

Create a trigger to monitor the **Update operation** on the Reserves table. *If* the modified tuple sid and bid attribute values do not exist in the Sailors and Boats tables, **roll back** the operation to modify the record.

Create trigger `referential_integrity_check`

Before Update of sid,bid on Reserves

Referencing NEW as N

For Each Row

Event

When (not (exists(Select * From Sailors Where sid = N.sid)
and
(exists(Select * From Boats Where bid = N.bid))
)

Rollback;

Condition

Action



Rule 5

Create a trigger to monitor the **Update operation** on the Sailors table. *If* there is a tuple in the Reserves table that references the pre modified sid value as a foreign key, **roll back** this modification operation.

```
Create trigger sailors_sid_update
Before Update of sid on Sailors
Referencing Old as O
For Each Row
When (exists(Select * From Reserves
              Where sid = O.sid))
Rollback;
```



When Not To Use Triggers

- Triggers were used earlier for tasks such as
 - Maintaining summary data (e.g., total salary of each department)
 - Replicating databases by recording changes to special relations (called **change** or **delta** relations) and having a separate process that applies the changes over to a replica
- There are better ways of doing these now:
 - Databases today provide built in materialized view facilities to maintain summary data
 - Databases provide built-in support for replication
- Encapsulation facilities can be used instead of triggers in many cases
 - Define methods to update fields
 - Carry out actions as part of the update methods instead of through a trigger



When Not To Use Triggers (Cont.)

- Risk of unintended execution of triggers, for example, when
 - Loading data from a backup copy
 - Replicating updates at a remote site
 - Trigger execution can be disabled before such actions.
- Other risks with triggers:
 - Error leading to failure of critical transactions that set off the trigger
 - Cascading execution



Execution of Rules

- *Immediate execution*
- *Deferred execution*
- *Decoupled or detached mode*
- *Cascading trigger*
 - Control nested execution of rules
 - Prevent nontermination
 - Triggering graph
 - Specify the upper limit of cascading times
 - So triggers should be used reasonably



Implementation of ECA

- Loosely coupling
- Tightly coupling (DB2, Oracle, etc.)
- Nested method

The rules are nested into transaction and executed by DBMS as a part of the transaction.

- Grafting method
- Query modification method



Summary

- SQL allows specification of rich integrity constraints (ICs): attribute-based, tuple-based CHECK and assertions (table-independent).
- CHECK constraints are activated only by modifications of the table they are based on, ASSERTIONS are activated by any modification that can possibly violate them.
- Choice of the most appropriate method for a particular IC is up to the DBA.
- Triggers respond to changes in the database. Can also be used to represent ICs.